OpenReporting
Raporlama ve gözlemlenebilirlik platformu

Kurulum Dokümanı

Linux · Docker · Kubernetes

Sürüm 1.1.0 · 9 Kasım 2026 · Ubuntu, Debian, RHEL, Oracle Linux, Rocky, Alma, SUSE

Bu dokümanda

- Kurulumdan önce hazırlanacaklar (kontrol listesi)
- Hangi yöntem: Docker mı, Kubernetes mi?
- Docker: her dağıtım için sistem hazırlığı
- Docker: tek komutla kurulum (sudo ./kurulum.sh)
- Kurumsal proxy ve SSL denetimi arkasında kurulum
- Kubernetes: küme, PostgreSQL, imaj, Secret, overlay
- Kurulum sonrası ilk ayarlar ve doğrulama
- Sorun giderme ve hızlı komut kartları

Örnek adlar (openreporting.example.com, srv-01, yonetici) temsilidir. Doküman parola, anahtar veya kurum bilgisi içermez.

İçindekiler

1. Önce okuyun . . . 3

1.1 Hangi yöntemi seçmeliyim? . . . 3

1.2 Desteklenen Linux dağıtımları . . . 3

2. Kurulumdan önce hazırlayın (kontrol listesi) . . . 4

3. Docker ile kurulum . . . 5

3.1 Sistemi hazırlama — Ubuntu / Debian / Kali . . . 5

3.2 Sistemi hazırlama — RHEL / Oracle Linux / Rocky / Alma . . . 5

3.3 Sistemi hazırlama — SUSE (SLES / openSUSE Leap) . . . 6

3.4 Kodu sunucuya alma . . . 6

3.5 Kurum sertifikasını yerleştirme (varsa) . . . 6

3.6 Kurulum: tek komut . . . 6

3.7 Soru sormadan kurulum (otomasyon / Ansible) . . . 7

4. Kurumsal proxy ve SSL denetimi arkasında kurulum . . . 7

4.1 Paket yöneticisi için proxy . . . 7

4.2 Docker servisi ve imaj derlemesi için proxy . . . 7

4.3 SSL denetimi (HTTPS'i yeniden imzalayan cihaz) . . . 8

5. Kubernetes ile kurulum . . . 8

5.1 Gereksinimler . . . 9

5.2 Küme yoksa: tek düğümlü k3s (isteğe bağlı) . . . 9

5.3 PostgreSQL hazırlığı . . . 9

5.4 İmajı derleme . . . 10

5.5 Namespace ve Secret'lar . . . 10

5.6 Kurum overlay'i . . . 11

5.7 Farklı Ingress controller kullanılıyorsa . . . 12

5.8 Uygulama ve ilk yönetici . . . 12

5.9 Kubernetes'te doğrulama ve işletim . . . 12

5.10 Docker kurulumundan Kubernetes'e geçiş . . . 12

6. Kurulumdan sonra (her iki yöntem) . . . 13

6.1 Kurulum günü kabul kontrolleri . . . 13

7. Sorun giderme (kurulum) . . . 14

Ek A. Docker hızlı komut kartı . . . 15

Ek B. Kubernetes hızlı komut kartı . . . 16

Ek C. Kurulum günü akışı (Docker) . . . 17

1. Önce okuyun

Bu doküman yalnız kurulumu anlatır: OpenReporting'i bir Linux sunucuya **Docker** ile veya bir **Kubernetes** kümesine kurmak için gereken her adımı, sırasıyla ve kopyalanabilir komutlarla verir. Uygulamanın kullanımı ve tüm ayarlar "Teknik Doküman"dadır.

1.1 Hangi yöntemi seçmeliyim?

	Docker (önerilen)	Kubernetes
Ne zaman	Tek sunucu. Kurumların büyük çoğunluğu için yeterli.	Kurumda zaten bir Kubernetes kümesi var ve yüksek erişilebilirlik / yatay ölçekleme isteniyor.
Uygulama veritabanı	SQLite (sunucuda, ek kurulum yok)	PostgreSQL 14+ (ayrıca hazırlanır)
Kurulum	`sudo ./kurulum.sh` — tek komut	İmaj + Secret + overlay + `kubectl apply`
Süre (internet hızına göre)	15–30 dakika	1–3 saat (küme hazırsa)
Yedek	Otomatik, her gece	PostgreSQL yedeği + paylaşımlı volume yedeği (kurum araçlarıyla)
Test kapsamı	Her değişiklikte CI'da uçtan uca kurulum (Ubuntu 24.04) + 14 dağıtım imajında Docker kurulumu	Her değişiklikte CI'da kind kümesinde 2 web + 2 worker + PostgreSQL

Not: Karar veremiyorsanız **Docker** ile başlayın. İleride Kubernetes'e geçmek gerekirse `migrate-to-postgres` komutu tüm veriyi taşır (Bölüm 5.10).

1.2 Desteklenen Linux dağıtımları

Dağıtım	Sürüm	Docker kurulumu (kurulum.sh)	CI'da test edilen	Not
Ubuntu	22.04, 24.04	Otomatik (apt, Docker resmi deposu)	22.04, 24.04 · 24.04'te uçtan uca kurulum	Önerilen
Debian	12, 13	Otomatik (apt)	12, 13	
Kali	rolling	Otomatik (apt, Debian kanalı)	rolling	Laboratuvar/deneme
Red Hat Enterprise Linux	8, 9, 10	Otomatik (dnf)	9 (UBI imajı)	Abonelik kayıtlı olmalı
Oracle Linux	8, 9, 10	Otomatik (dnf)	8, 9, 10 · 9'da Podman kaldırma	Podman varsa onayla kaldırılır
Rocky Linux / AlmaLinux	8, 9	Otomatik (dnf)	9	
Fedora	güncel	Otomatik (dnf)	42	Deneme amaçlı
SUSE: SLES / openSUSE Leap	15.x	Otomatik (zypper, dağıtım paketi)	openSUSE Leap 15.6	SLES'te önce Containers modülü
Diğer	—	Elle	—	Docker 24+ ve Compose v2 kurulunca betik çalışır

CI testi: Docker'ın resmi depodan kurulduğu, `docker`, `docker compose`, `buildx`, `openssl`, `git` ve cron'un hazır olduğu her değişiklikte bu dağıtımların konteyner imajlarında doğrulanır. Uygulamanın baştan sona kurulumu (servisler, HTTPS, giriş, yedek, çıkış kilidi, ikinci çalıştırma) Ubuntu 24.04 sunucuda test edilir.

Gereken işlemci mimarisi **x86_64**'tür (ARM64 imajı derlenebilir ama test edilmez). Windows Server ve Docker Desktop sunucu için kullanılmaz.

2. Kurulumdan önce hazırlayın (kontrol listesi)

Kurulum günü beklememek için aşağıdakileri önceden tamamlayın. Her satırın sahibi genellikle farklı bir ekiptir (sistem, ağ, güvenlik, veritabanı).

#	Hazırlık	Ayrıntı	Kimden
1	Sunucu	Docker için en az **4 vCPU, 8 GB RAM, 50 GB SSD** (deneme: 2 vCPU / 4 GB / 20 GB). Disk yerel olmalı (NFS/SMB değil). Root veya sudo yetkisi.	Sistem
2	DNS adı	Ör. `openreporting.example.com` → sunucunun IP'si. IP ile de çalışır, ama sertifika ve kullanıcı alışkanlığı için DNS önerilir.	Ağ / DNS
3	TLS sertifikası	Bu ad için kurum CA'sından sertifika: zincir PEM (`cert.pem`) ve özel anahtar PEM (`key.pem`). Yoksa kurulum kendinden imzalı üretir (tarayıcı uyarı verir).	Güvenlik / PKI
4	Kurulum sırasında internet	Sunucudan şu adreslere HTTPS (443) çıkışı: dağıtım depoları, `download.docker.com`, `registry-1.docker.io`, `production.cloudflare.docker.com`, `pypi.org`, `files.pythonhosted.org`, `github.com`. **Kurulumdan sonra internet gerekmez.**	Ağ / Güvenlik
5	Kurumsal proxy / SSL denetimi	Varsa proxy adresi (`http://proxy.example.com:3128`) ve HTTPS'i yeniden imzalayan kök sertifika (PEM). Bkz. Bölüm 4.	Ağ / Güvenlik
6	Gelen bağlantı	Kullanıcı ağlarından sunucuya **443/tcp** (ve yönlendirme için 80/tcp).	Ağ
7	Giden bağlantı (kalıcı)	Sunucudan veri kaynaklarına (Zabbix API, veritabanları …), SMTP'ye, LDAP'a, iç DNS ve NTP'ye erişim. Bu ağların listesi (CIDR) çıkış kilidi için istenir.	Ağ
8	Saat senkronu	Sunucuda NTP açık olmalı (iki adımlı giriş kodları saate duyarlıdır). İnternetsiz ortamda iç NTP sunucusu adresi.	Sistem
9	Kaynak kod erişimi	Depo herkese açıktır (MIT lisansı): `https://github.com/sergenbasakcii/openreporting`. Hesap veya token gerekmez; yalnız `github.com`'a HTTPS (443) çıkışı.	Ağ
10	Veri kaynağı hesapları	Her kaynak için **yalnız okuma** yetkili hesap/token: Zabbix API token, veritabanı kullanıcısı vb. Kurum CA'sı kullanılıyorsa CA PEM'i.	Uygulama sahipleri
11	E-posta (isteğe bağlı)	SMTP sunucusu, port (587/465), gönderen adresi, gerekiyorsa kullanıcı/parola, izinli alıcı alan adları.	E-posta ekibi
12	LDAP / AD (isteğe bağlı)	LDAPS adresi, salt okunur servis hesabı, arama tabanı, grup → rol eşlemesi (yönetici/editör/izleyici grupları), DC CA sertifikası.	Dizin ekibi
13	Yönetici	İlk yönetici kullanıcı adı ve en az **14 karakterlik** parola; telefonunda doğrulayıcı uygulama (Microsoft/Google Authenticator, FreeOTP).	Siz
14	Kasa	Şifreleme anahtarının ve MFA kurtarma kodlarının saklanacağı yer (parola yöneticisi / kurum kasası).	Güvenlik

> **Dikkat:** Ağ ekibine 4. ve 7. satırları önceden iletin; kurulum günü en sık takılan nokta kapalı giden bağlantılardır.

3. Docker ile kurulum

Docker kurulumu her dağıtımda aynı dört adımdır: **(1) sistemi hazırla → (2) kodu al → (3) gerekirse sertifikayı koy → (4)** `sudo ./kurulum.sh`. Docker kurulu değilse `kurulum.sh` onu Docker'ın resmi deposundan kendisi kurar. Bu bölümdeki "Docker'ı elle kurmak" komutları yalnız Docker'ı ayrıca kurmak isteyenler içindir.

3.1 Sistemi hazırlama — Ubuntu / Debian / Kali

```
sudo apt-get update && sudo apt-get -y upgrade
sudo apt-get install -y git openssl ca-certificates curl
sudo timedatectl set-ntp true && timedatectl        # "System clock synchronized: yes" görünmeli
sudo hostnamectl set-hostname openreporting          # isteğe bağlı
```

Güvenlik duvarı `ufw` açıksa `kurulum.sh` 80/443'ü kendisi açar. Elle: `sudo ufw allow 80/tcp && sudo ufw allow 443/tcp`.

Docker'ı elle kurmak (isteğe bağlı) — Ubuntu

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
REPO=https://download.docker.com/linux/ubuntu
CODENAME="$(. /etc/os-release && echo "$VERSION_CODENAME")"
ARCH="$(dpkg --print-architecture)"
echo "deb [arch=$ARCH signed-by=/etc/apt/keyrings/docker.asc] $REPO $CODENAME stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

Debian ve Kali'de adreslerde `ubuntu` yerine `debian` yazın; Kali'de sürüm adı olarak `bookworm` kullanın.

3.2 Sistemi hazırlama — RHEL / Oracle Linux / Rocky / Alma

```
sudo dnf -y update
sudo dnf -y install git openssl curl tar cronie chrony
sudo systemctl enable --now chronyd crond
timedatectl                                          # "System clock synchronized: yes" görünmeli
```

- **RHEL:** sunucu Red Hat aboneliğine kayıtlı olmalı (`sudo subscription-manager status`).
- **Podman:** bu dağıtımlarda varsayılan gelir ve Docker ile çakışır. `kurulum.sh` kaldırmak için onay ister. Elle: `sudo dnf -y remove podman buildah runc`.
- **SELinux:** açık (enforcing) kalabilir; kapatmayın. Bağlanan dosyalar `:z` etiketiyle ayarlıdır.
- **firewalld:** açıksa `kurulum.sh` 80/443'ü kendisi açar. Elle: `sudo firewall-cmd --permanent --add-service=http --add-service=https && sudo firewall-cmd --reload`.
- İnternetsiz ortamda iç NTP: `/etc/chrony.conf` içine `server ntp.example.com iburst` yazıp `sudo systemctl restart chronyd`.

Docker'ı elle kurmak (isteğe bağlı) — RHEL ailesi

```
# Oracle Linux, Rocky, Alma, CentOS Stream: "centos" deposu · RHEL: "rhel" deposu
sudo curl -fsSL https://download.docker.com/linux/centos/docker-ce.repo \
  -o /etc/yum.repos.d/docker-ce.repo
sudo dnf -y install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

3.3 Sistemi hazırlama — SUSE (SLES / openSUSE Leap)

```
# Yalnız SLES: Containers modülünü açın (sürümü kendi sisteminize göre yazın)
sudo SUSEConnect -p sle-module-containers/15.6/x86_64
sudo zypper refresh && sudo zypper -n update
sudo zypper -n install git openssl cron chrony
sudo systemctl enable --now chronyd
```

SUSE'de Docker dağıtımın kendi paketinden (`docker`, `docker-compose`) kurulur; sürüm 24 veya üstü olmalıdır (`docker version`).

3.4 Kodu sunucuya alma

Depo herkese açıktır; GitHub hesabı, token veya anahtar gerekmez:

```
git clone https://github.com/sergenbasakcii/openreporting.git ~/openreporting
cd ~/openreporting
```

Sunucu GitHub'a çıkamıyorsa depo sayfasındaki **Code → Download ZIP** ile indirip sunucuda `~/openreporting` klasörüne açın.

3.5 Kurum sertifikasını yerleştirme (varsa)

```
cd ~/openreporting
# sunucu sertifikası + ara sertifikalar:
cp /yol/sertifika-zinciri.pem deploy/docker/certs/cert.pem
# şifresiz özel anahtar:
cp /yol/ozel-anahtar.pem deploy/docker/certs/key.pem
```

- `cert.pem` önce sunucu sertifikası, sonra ara (intermediate) sertifikaları içermelidir.
- Özel anahtar parola korumalıysa önce çözün: `openssl pkey -in sifreli.key -out deploy/docker/certs/key.pem`.
- Uyum kontrolü (iki çıktı aynı olmalı): `openssl x509 -noout -pubkey -in deploy/docker/certs/cert.pem | sha256sum` ve `openssl pkey -pubout -in deploy/docker/certs/key.pem | sha256sum`.
- Sertifika yoksa bu adımı atlayın; `kurulum.sh` kendinden imzalı sertifika üretir. Kurum sertifikası sonra gelince dosyaları koyup `sudo ./kurulum.sh` yeniden çalıştırılır.

3.6 Kurulum: tek komut

```
cd ~/openreporting
sudo ./kurulum.sh
```

Betik yalnız şunları sorar:

Soru	Ne yazılır	Not
Docker kurulsun mu?	Enter (Evet)	Yalnız Docker yoksa sorulur; resmi depodan kurar. RHEL ailesinde Podman kaldırma onayı da istenir.
Kullanıcıların tarayıcıya yazacağı adres	`openreporting.example.com`	Varsayılanı (sunucu adı) önerir. `https://` yazmayın.
HTTPS portu	Enter (443)	443 başka bir uygulamada kullanılıyorsa farklı port verin (ör. 8443).
Yönetici kullanıcı adı	`yonetici`	
Parola (iki kez)	En az 14 karakter	Ekranda görünmez, betikten geçmez.

Soru	Ne yazılır	Not
İç ağlar	`10.0.0.0/8` `192.168.5.10/32`	Uygulamanın erişeceği ağlar (veri kaynakları, SMTP, LDAP, DNS, NTP). Diğer her yere çıkış kapatılır. **Emin değilseniz boş bırakın**; sonra eklenir.

Betiğin kendiliğinden yaptıkları: Docker kurulumu ve sürüm kontrolü, güvenlik duvarında 80/443, ayar dosyası (root:10001, 0640), sertifika ve izinleri, imaj derleme ve başlatma (ilk seferde 5–15 dakika), ilk yönetici, şifreleme anahtarının `/root/openreporting-anahtar/` içine kopyası, gece 02:30 yedeği (`/etc/cron.d/openreporting-yedek`, 14 gün saklama), isteğe bağlı çıkış kilidi (systemd birimi), saat ve `doctor` kontrolü.

Başarılı kurulum şu satırla biter:

```
== Kurulum tamam ==
  Adres     : https://openreporting.example.com
  Sonra     : Yönetim → Sistem ayarları (e-posta, LDAP) ve Yönetim → Veri kaynakları
  Durum     : sudo ./kurulum.sh durum      Güncelleme: sudo ./kurulum.sh guncelle
```

> **Not:** Betik tekrar çalıştırılabilir. Yarıda kalırsa (ör. ağ kesintisi) sorunu giderip aynı komutu yeniden çalıştırın; var olan ayar, sertifika, kullanıcı ve veriye dokunmaz.

3.7 Soru sormadan kurulum (otomasyon / Ansible)

```
# parola ekrana ve komut geçmişine yazılmaz:
read -rs -p 'Yönetici parolası: ' OR_PAROLA; echo; export OR_PAROLA
sudo -E OR_EVET=1 OR_ADRES=openreporting.example.com OR_YONETICI=yonetici \
     OR_IC_AGLAR="10.0.0.0/8" ./kurulum.sh
unset OR_PAROLA
```

Ansible gibi araçlarda parolayı `no_log: true` ile ortam değişkeni olarak verin. Diğer değişkenler: `OR_HTTPS_PORT` (443), `OR_SERTIFIKA` + `OR_ANAHTAR` (kurum PEM dosyaları). `OR_IC_AGLAR` boş bırakılırsa çıkış kilidi atlanır.

4. Kurumsal proxy ve SSL denetimi arkasında kurulum

Sunucu internete bir proxy üzerinden çıkıyorsa veya güvenlik cihazı HTTPS'i kendi sertifikasıyla yeniden imzalıyorsa (SSL inspection), kurulumdan **önce** şu ayarları yapın.

4.1 Paket yöneticisi için proxy

```
# Ubuntu/Debian
echo 'Acquire::https::Proxy "http://proxy.example.com:3128";' | sudo tee /etc/apt/apt.conf.d/95proxy
# RHEL ailesi (/etc/dnf/dnf.conf içinde [main] altına)
echo 'proxy=http://proxy.example.com:3128' | sudo tee -a /etc/dnf/dnf.conf
# kurulum betiği ve curl/git için (aynı oturumda)
export https_proxy=http://proxy.example.com:3128
export http_proxy=http://proxy.example.com:3128
export no_proxy=localhost,127.0.0.1
```

Ortam değişkenlerinin `sudo` ile korunması için kurulumu `sudo -E ./kurulum.sh` ile çalıştırın.

4.2 Docker servisi ve imaj derlemesi için proxy

Docker kurulduktan sonra (betik Docker'ı kurup durursa veya Docker'ı elle kurduysanız) şu iki dosyayı yazın, sonra `sudo ./kurulum.sh` çalıştırın:

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf <<'EOF'
[Service]
Environment="HTTP_PROXY=http://proxy.example.com:3128" "HTTPS_PROXY=http://proxy.example.com:3128"
Environment="NO_PROXY=localhost,127.0.0.1,172.30.88.0/24"
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker

# İmaj derlemesindeki pip indirmeleri için (root kullanıcısının Docker istemcisi)
sudo mkdir -p /root/.docker
sudo tee /root/.docker/config.json <<'EOF'
{
  "proxies": {
    "default": {
      "httpProxy": "http://proxy.example.com:3128",
      "httpsProxy": "http://proxy.example.com:3128",
      "noProxy": "localhost,127.0.0.1"
    }
  }
}
EOF
```

4.3 SSL denetimi (HTTPS'i yeniden imzalayan cihaz)

Belirti: derlemede `CERTIFICATE_VERIFY_FAILED` veya paket yöneticisinde "SSL certificate problem". Cihazın kök sertifikasını (yalnız açık kısmı, PEM) iki yere koyun:

```
# 1) İşletim sistemi (paket yöneticisi, curl, Docker çekme işlemleri)
#    Ubuntu/Debian:
sudo cp kurum-kok.pem /usr/local/share/ca-certificates/kurum-kok.crt && sudo update-ca-certificates
#    RHEL ailesi:
sudo cp kurum-kok.pem /etc/pki/ca-trust/source/anchors/kurum-kok.pem && sudo update-ca-trust
sudo systemctl restart docker

# 2) İmaj derlemesi: yalnız derlemede güvenilir, çalışan imaja girmez
#    (paketler yine özetleriyle doğrulanır)
cp kurum-kok.pem ~/openreporting/deploy/docker/build-ca/kurum-kok.crt
```

5. Kubernetes ile kurulum

Bu yol, kurumda bir Kubernetes kümesi olduğunda veya yüksek erişilebilirlik istendiğinde seçilir. Mimari: Ingress (TLS) → web pod'ları (2–10, otomatik ölçekleme) + worker pod'ları (1 aktif + 1 sıcak yedek) → PostgreSQL; rapor arşivi paylaşımlı bir volume'dedir.

5.1 Gereksinimler

Bileşen	Gereksinim	Not
Kubernetes	1.27+	Kurum kümesi (OpenShift hariç), RKE2, k3s, kubeadm, EKS/AKS/GKE
Ingress controller	ingress-nginx (`ingressClassName: nginx`, namespace `ingress-nginx`)	Başka controller kullanılıyorsa Bölüm 5.7'deki uyarlamalar gerekir
metrics-server	Kurulu	Web pod'larının otomatik ölçeklemesi (HPA) için. k3s'te hazır gelir
NetworkPolicy	CNI uygulamalı (Calico, Cilium, k3s'in yerleşik denetleyicisi)	Uygulamayan CNI'da web pod'larına başka pod'lar da erişebilir
PostgreSQL	14+ ve yalnız bu uygulamaya ait veritabanı + rol	Yönetilen servis, operatör (CloudNativePG) veya ayrı bir VM (Bölüm 5.3)
Depolama	ReadWriteMany StorageClass (NFS, CephFS, Longhorn RWX ...), 50 GB	Tek düğümlü kümede ReadWriteOnce yeterlidir
İmaj deposu	Kurum registry'si (Harbor, Nexus, GitLab ...)	Tek düğümlü k3s'te gerekmez (imaj doğrudan yüklenir)
Yönetim makinesi	`kubectl` 1.27+ (kustomize dahildir), `docker`, `git`, `openssl`	İmajı derlemek için Docker gerekir

Not: Kubernetes topluluğu ingress-nginx projesinin bakımının sona erdiğini duyurmuştur. Manifestler ve CI testleri ingress-nginx ile yapılır; kurumunuzun standart controller'ı farklıysa Bölüm 5.7'deki iki noktayı uyarlayın.

5.2 Küme yoksa: tek düğümlü k3s (isteğe bağlı)

Kurumda küme yoksa ve Kubernetes isteniyorsa tek bir Linux sunucuda (4 vCPU, 8 GB, 80 GB) k3s ile başlanabilir. Bu yol CI'da test edilmez; CI kind kümesini kullanır.

```
# k3s (Traefik kapalı: ingress-nginx kullanılacak)
curl -sfL https://get.k3s.io | sudo INSTALL_K3S_EXEC="--disable traefik" sh -
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown "$(id -u):$(id -g)" ~/.kube/config
kubectl get nodes                                    # STATUS Ready

# ingress-nginx (CI'da test edilen sürüm); k3s'in servicelb'si 80/443'ü düğüm IP'sinde açar
INGRESS=https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.1
kubectl apply -f "$INGRESS/deploy/static/provider/cloud/deploy.yaml"
kubectl -n ingress-nginx wait --for=condition=ready pod \
  -l app.kubernetes.io/component=controller --timeout=300s
```

k3s'in pod ağı varsayılan **10.42.0.0/16**'dır (Bölüm 5.6'da gerekir). Depolama olarak hazır gelen `local-path` (ReadWriteOnce) kullanılır. RHEL ailesinde firewalld açıksa k3s belgelerindeki port kurallarını uygulayın veya firewalld'ı kapatın (kurum politikanıza göre).

5.3 PostgreSQL hazırlığı

Yönetilen bir PostgreSQL veya kurum DBA ekibinin verdiği bir veritabanı varsa yalnız şu bilgiler istenir: adres, port, veritabanı adı, kullanıcı, parola, TLS CA sertifikası. Kendi VM'inizde kurmak için:

```
# Ubuntu 24.04 (PostgreSQL 16)
sudo apt-get install -y postgresql
# RHEL / Oracle Linux / Rocky / Alma 8-9 (PostgreSQL 16 modülü)
sudo dnf -y module enable postgresql:16 && sudo dnf -y install postgresql-server
sudo postgresql-setup --initdb && sudo systemctl enable --now postgresql

# Rol ve veritabanı (parola sorulur, komut geçmişine yazılmaz)
sudo -u postgres createuser --pwprompt openreporting
sudo -u postgres createdb -O openreporting openreporting
```

Uzak erişim: `postgresql.conf` içinde `listen_addresses = '*'`; `pg_hba.conf` sonuna küme düğümlerinin ağı için `hostssl openreporting openreporting 10.0.0.0/8 scram-sha-256` satırı; TLS için sunucu sertifikası (`ssl = on`, `ssl_cert_file`, `ssl_key_file`). Sonra `sudo systemctl restart postgresql` ve güvenlik duvarında 5432/tcp yalnız küme düğümlerine.

Bağlantı adresi biçimi (bağlantı TLS ile şifrelenir):

```
postgresql://openreporting:PAROLA@pg.example.com:5432/openreporting?sslmode=require
```

Parolada `@ : / ? # &` gibi karakterler varsa URL kodlaması gerekir (ör. `@` → `%40`); en kolayı parolayı harf ve rakamdan seçmektir. Sunucu sertifikasını da doğrulamak (`sslmode=verify-full`) için veritabanı CA dosyasının pod'lara ayrıca bağlanması gerekir; repodaki manifestler bunu içermez, ilk kurulumda `require` yeterlidir.

5.4 İmajı derleme

```
# kod (Bölüm 3.4):
git clone https://github.com/sergenbasakcii/openreporting.git ~/openreporting
cd ~/openreporting

# A) Kurum registry'si
docker build -t registry.example.com/openreporting/openreporting:1.1.0 .
docker login registry.example.com
docker push registry.example.com/openreporting/openreporting:1.1.0

# B) Tek düğümlü k3s: registry olmadan, imajı doğrudan k3s'e yükleyin
docker build -t localhost/openreporting:1.1.0 .
docker save localhost/openreporting:1.1.0 -o /tmp/openreporting.tar
sudo k3s ctr images import /tmp/openreporting.tar
```

Docker yoksa Bölüm 3.1–3.3'teki "Docker'ı elle kurmak" komutlarıyla kurun. Proxy/SSL denetimi varsa Bölüm 4 imaj derlemesi için de geçerlidir.

5.5 Namespace ve Secret'lar

Parolalar komut satırına yazılmasın diye değerler 0600 izinli dosyalardan okunur.

```
kubectl create namespace openreporting

# 1) Veritabanı adresi — dosyada TIRNAK KULLANMAYIN (env-file değerleri olduğu gibi okunur)
install -m 0600 /dev/null ~/or-db.env
# dosyaya tek satır yazın (tırnaksız), adres örneği alttadır:
#   OPENREPORTING_DATABASE_URL=<adres>
#   postgresql://openreporting:PAROLA@pg.example.com:5432/openreporting?sslmode=require
nano ~/or-db.env
kubectl -n openreporting create secret generic openreporting-secrets \
  --from-env-file="$HOME/or-db.env"

# 2) Şifreleme anahtarı — bir kez üretilir, tüm pod'lar aynısını kullanır, KASAYA konur
# k3s yolunda: IMAGE=localhost/openreporting:1.1.0
IMAGE=registry.example.com/openreporting/openreporting:1.1.0
docker run --rm "$IMAGE" python -c \
  'from cryptography.fernet import Fernet as F; print(F.generate_key().decode(), end="")' \
  > ~/secret.key
chmod 600 ~/secret.key
kubectl -n openreporting create secret generic openreporting-key \
  --from-file=secret.key="$HOME/secret.key"

# 3) TLS sertifikası (Bölüm 3.5'teki dosyalar)
kubectl -n openreporting create secret tls openreporting-tls --cert=cert.pem --key=key.pem

# Anahtar kopyasını kasaya taşıdıktan sonra sunucudaki dosyaları silin
shred -u ~/secret.key ~/or-db.env
```

> **Dikkat:** Şifreleme anahtarı kaybolursa kayıtlı veri kaynağı, SMTP ve LDAP parolaları ve MFA kayıtları açılamaz. `shred` komutundan önce kasadaki kopyayı doğrulayın.

5.6 Kurum overlay'i

Repodaki `deploy/kubernetes/base` olduğu gibi kalır; kuruma özel değerler bir overlay'de tutulur:

```
mkdir -p deploy/kubernetes/overlays/prod
cat > deploy/kubernetes/overlays/prod/kustomization.yaml <<'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: openreporting
resources: [../../base]
images:
  # A) registry:  newName: registry.example.com/openreporting/openreporting
  # B) k3s:       newName: localhost/openreporting
  - name: registry.example.com/openreporting/openreporting
    newName: registry.example.com/openreporting/openreporting
    newTag: "1.1.0"
patches:
  - target: {kind: ConfigMap, name: openreporting-config}
    patch: |
      - op: replace
        path: /data/OPENREPORTING_PUBLIC_URL
        value: "https://openreporting.example.com"
      # ZORUNLU: kümenin pod ağı (k3s 10.42.0.0/16; Calico çoğunlukla 192.168.0.0/16)
      - op: replace
        path: /data/OPENREPORTING_FORWARDED_ALLOW_IPS
        value: "10.42.0.0/16"
  - target: {kind: Ingress, name: "openreporting(-login)?"}
    patch: |
      - {op: replace, path: /spec/rules/0/host, value: openreporting.example.com}
      - {op: replace, path: /spec/tls/0/hosts/0, value: openreporting.example.com}
  # RWX StorageClass adı (k3s tek düğüm: aşağıdaki iki satırın yerine accessModes: [ReadWriteOnce])
  - target: {kind: PersistentVolumeClaim, name: openreporting-archive}
    patch: |
      - {op: add, path: /spec/storageClassName, value: nfs-client}
EOF
```

k3s tek düğüm için son yamayı şununla değiştirin:

```
  - target: {kind: PersistentVolumeClaim, name: openreporting-archive}
    patch: |
      - {op: replace, path: /spec/accessModes, value: [ReadWriteOnce]}
```

Pod ağını öğrenmek için: `kubectl get nodes -o jsonpath='{.items[*].spec.podCIDR}'` (düğüm başına alt ağlar; hepsini kapsayan üst ağı yazın).

5.7 Farklı Ingress controller kullanılıyorsa

- `ingressClassName: nginx` değerini kendi sınıfınızla değiştirin (overlay'de `/spec/ingressClassName` yaması).
- `networkpolicy.yaml` web pod'larına yalnız `ingress-nginx` namespace'indeki controller pod'larını alır; controller'ınızın namespace ve etiketlerini bu kurala yazın.
- Giriş sayfası oran sınırı ve HTTPS yönlendirmesi ingress-nginx annotation'larıyla yapılır; diğer controller'larda eşdeğer ayarı kullanın.

5.8 Uygulama ve ilk yönetici

```
kubectl apply -k deploy/kubernetes/overlays/prod
kubectl -n openreporting rollout status deploy/openreporting-web --timeout=600s
kubectl -n openreporting rollout status deploy/openreporting-worker --timeout=600s
kubectl -n openreporting get pods                    # web ×2 ve worker ×2: Running, READY 1/1

# Veritabanı ön kontrolü (salt okunur)
kubectl -n openreporting exec deploy/openreporting-web -- openreporting check-postgres

# İlk yönetici (parola sorulur, en az 14 karakter)
kubectl -n openreporting exec -it deploy/openreporting-web -- \
  openreporting create-user yonetici --role admin
```

DNS: `openreporting.example.com` adını Ingress controller'ın dış IP'sine yönlendirin (`kubectl -n ingress-nginx get svc ingress-nginx-controller`; k3s'te düğümün IP'si).

5.9 Kubernetes'te doğrulama ve işletim

```
kubectl -n openreporting exec deploy/openreporting-web -- openreporting doctor
# worker'lardan biri "waiting as standby" yazar:
kubectl -n openreporting logs -l app.kubernetes.io/component=worker --tail=20
# TARGETS bir yüzde göstermeli (metrics-server çalışıyor):
kubectl -n openreporting get hpa
curl -sk -o /dev/null -w '%{http_code}\n' https://openreporting.example.com/api/health   # 200

# Güncelleme
NEW=registry.example.com/openreporting/openreporting:1.1.1
git pull && docker build -t "$NEW" . && docker push "$NEW"
kubectl -n openreporting set image deploy/openreporting-web web="$NEW"
kubectl -n openreporting set image deploy/openreporting-worker worker="$NEW"
```

Yedek: PostgreSQL'in kurum yedeği (PITR önerilir) + arşiv volume'ünün yedeği + kasadaki anahtar; üçü aynı ana ait olmalıdır.

5.10 Docker kurulumundan Kubernetes'e geçiş

1. Docker sunucusunda yedek alın: `sudo ./kurulum.sh yedek`, sonra `docker compose stop web worker`.
2. PostgreSQL adresini `/root/pg-target.env` dosyasına (0600, tırnaklı: `OPENREPORTING_DATABASE_URL='postgresql://…'`) yazın ve aşağıdaki komutla taşıyın; çıktı `"ok": true` olmalıdır.
3. `openreporting-key` Secret'ını Docker sunucusundaki anahtardan oluşturun (`/root/openreporting-anahtar/secret.key`), yeni anahtar üretmeyin.

4. Rapor arşivi ve logoları (`archive`, `logos` klasörleri) arşiv volume'üne kopyalayın (sahip uid/gid 10001).
5. Kubernetes kurulumunu başlatın, eski bir raporu indirip `openreporting verify-audit` çalıştırın.

```
cd ~/openreporting
sudo bash -c 'set -a; . /root/pg-target.env; set +a
  docker compose run --rm --no-deps -e OPENREPORTING_DATABASE_URL web \
    openreporting migrate-to-postgres --services-stopped \
      --source /var/lib/openreporting/state.sqlite3'
```

6. Kurulumdan sonra (her iki yöntem)

1. Tarayıcıda adresi açın, yöneticiyle girin; telefonla QR'ı okutup 6 haneli kodu yazın; **kurtarma kodlarını kasaya kaydedin**.
2. **Şifreleme anahtarı:** Docker'da `/root/openreporting-anahtar/secret.key` dosyasını sunucu dışına (kasa) alın: `sudo cat /root/openreporting-anahtar/secret.key`.
3. **Yönetim → Sistem ayarları → E-posta:** SMTP bilgileri, izinli alıcı alan adları, **Bağlantıyı test et**.
4. **Yönetim → Sistem ayarları → LDAP** (kullanılıyorsa): adres, servis hesabı, arama tabanı, grup → rol eşlemesi, CA; test.
5. **Yönetim → Veri kaynakları → Veri kaynağı ekle:** Zabbix vb., **Oluştur ve test et**.
6. **Yönetim → Kullanıcılar / Takımlar:** kullanıcılar ve takımlar; gerekiyorsa klasör izinleri.
7. Örnek bir dashboard veya akıllı dashboard ile **PDF önizle**; bir raporu **Şimdi çalıştır** ile e-postayla deneyin.
8. Docker'da çıkış kilidi kurulumda atlandıysa iç ağlar netleşince açın: `sudo OR_IC_AGLAR="10.0.0.0/8 192.168.5.10/32" ./kurulum.sh`.

6.1 Kurulum günü kabul kontrolleri

Kontrol	Docker	Kubernetes
Servisler	`sudo ./kurulum.sh durum` → hepsi ✓	`kubectl -n openreporting get pods` → hepsi Running
Sağlık	`curl -sk https://ADRES/api/health` → 200	Aynı
Kurulum denetimi	`docker compose exec web openreporting doctor`	`kubectl … exec deploy/openreporting-web -- openreporting doctor`
Saat	`timedatectl` → synchronized: yes	Düğümlerde aynı
Yedek	`sudo ./kurulum.sh yedek` → `backups/` altında yeni klasör	Kurum yedek aracında ilk iş başarılı
Anahtar	Kasadaki kopya doğrulandı	Kasadaki kopya doğrulandı
Giriş	Yönetici + MFA ile giriş	Aynı
Uçtan uca	Veri kaynağı testi yeşil; test raporu e-postayla geldi	Aynı

7. Sorun giderme (kurulum)

Belirti	Neden	Çözüm
`SSL certificate problem` / `CERTIFICATE_VERIFY_FAILED`	Proxy veya güvenlik cihazı HTTPS'i yeniden imzalıyor	Bölüm 4.3: kök sertifikayı sisteme ve `deploy/docker/build-ca/` içine ekleyin
`Could not resolve host` / zaman aşımı	DNS veya giden bağlantı kapalı	Bölüm 2, satır 4; proxy varsa Bölüm 4.1–4.2
git clone: `Could not resolve host: github.com` / zaman aşımı	GitHub'a çıkış kapalı	Ağ ekibinden `github.com:443`; ya da ZIP ile kurulum (Bölüm 3.4)
dnf: `podman … conflicts with containerd.io`	Podman kurulu	`sudo dnf -y remove podman buildah runc`, sonra tekrar
RHEL: `This system is not registered`	Abonelik yok	`sudo subscription-manager register`
Docker sürümü eski hatası	Dağıtım paketinden eski Docker	Eski paketleri kaldırın (`docker.io`, `docker-ce` dışındakiler), betik resmi depodan kurar
`Pool overlaps with other one on this address space`	Docker iç ağı (172.30.88.0/24) kurum ağıyla çakışıyor	`~/openreporting/.env` dosyasına `OPENREPORTING_SUBNET=172.31.200.0/24`, `OPENREPORTING_PROXY_IP=172.31.200.10`, `OPENREPORTING_WEB_IP=172.31.200.11`; tekrar çalıştırın
`port is already allocated` (80/443)	Sunucuda başka web sunucusu var	Onu durdurun veya farklı HTTPS portu verin
proxy: `cannot load certificate key`	Anahtar şifreli veya biçimi hatalı	Bölüm 3.5: şifreyi çözün, eşleşmeyi kontrol edin
Tarayıcıda 400 "Invalid host header"	Tarayıcıdaki adres kurulumda verilen adresten farklı	Ek adları `deploy/docker/openreporting.env` içinde `OPENREPORTING_ALLOWED_HOSTS` satırına yazıp `docker compose up -d`
MFA kodu kabul edilmiyor	Sunucu veya telefon saati kaymış	`timedatectl`; NTP'yi düzeltin
Kubernetes: pod `Şifreleme anahtarı bulunamadı`	`openreporting-key` Secret'ı yok	Bölüm 5.5 adım 2
Kubernetes: readiness başarısız, `/api/health` 503	PostgreSQL'e erişilemiyor	`check-postgres`; adres, `pg_hba.conf`, TLS, güvenlik duvarı
Kubernetes: MFA kurulumu "yalnız HTTPS"	`OPENREPORTING_FORWARDED_ALLOW_IPS` pod ağını kapsamıyor	Bölüm 5.6; ConfigMap'i düzeltip `kubectl rollout restart deploy/openreporting-web`
Kubernetes: PVC `Pending`	RWX StorageClass yok / adı yanlış	`kubectl get storageclass`; overlay'deki adı düzeltin (tek düğümde RWO)

Destek isterken şu çıktıları gönderin (parola ve anahtar içermez): `sudo ./kurulum.sh durum`, `docker compose logs --tail=200 web worker proxy` (Kubernetes'te `kubectl -n openreporting describe pods` ve `logs`), `cat /etc/os-release`, `docker version`.

Ek A. Docker hızlı komut kartı

```
# Kurulum (Ubuntu / Debian / RHEL / Oracle / Rocky / Alma / SUSE)
git clone https://github.com/sergenbasakcii/openreporting.git ~/openreporting
cd ~/openreporting && sudo ./kurulum.sh

# Günlük
sudo ./kurulum.sh durum                # servisler, worker, son yedek, çıkış kilidi
sudo ./kurulum.sh yedek                # hemen yedek
sudo ./kurulum.sh guncelle             # yedek → yeni sürüm → derle → kontrol
docker compose logs -f web worker      # günlükler
docker compose exec web openreporting doctor
docker compose exec web openreporting reset-password <kullanıcı>
docker compose exec web openreporting reset-mfa <kullanıcı> --reason lost-device
```

Ek B. Kubernetes hızlı komut kartı

```
kubectl create namespace openreporting
kubectl -n openreporting create secret generic openreporting-secrets \
  --from-env-file="$HOME/or-db.env"
kubectl -n openreporting create secret generic openreporting-key \
  --from-file=secret.key="$HOME/secret.key"
kubectl -n openreporting create secret tls openreporting-tls --cert=cert.pem --key=key.pem
kubectl apply -k deploy/kubernetes/overlays/prod
kubectl -n openreporting rollout status deploy/openreporting-web
kubectl -n openreporting exec -it deploy/openreporting-web -- \
  openreporting create-user yonetici --role admin
kubectl -n openreporting get pods,svc,ingress,pvc,hpa
kubectl -n openreporting exec deploy/openreporting-web -- openreporting doctor
```

Ek C. Kurulum günü akışı (Docker)

Saat	Adım	Süre
1	Kontrol listesi (Bölüm 2): sunucu, DNS, sertifika, giden bağlantılar	önceden
2	Sistem hazırlığı (Bölüm 3.1 / 3.2 / 3.3)	10 dk
3	Kodu alma (Bölüm 3.4)	5 dk
4	Sertifikayı koyma (Bölüm 3.5)	5 dk
5	`sudo ./kurulum.sh` (Bölüm 3.6)	15–30 dk
6	İlk giriş, MFA, anahtarı kasaya alma (Bölüm 6)	10 dk
7	E-posta, LDAP, veri kaynakları (Bölüm 6)	20–60 dk
8	Kabul kontrolleri (Bölüm 6.1)	15 dk